

❖ Functional Programming

1. Lambda Function

- A Lambda Function is a small anonymous (nameless) function written in a single line using the lambda keyword.
- A Lambda Function is a one-line function without a name.

Why do we use Lambda Functions?

- Short and simple functions
- Reduces code
- Used with map(), filter(), and reduce()
- Improves readability for simple operations

1) Normal Function

```
def square(x):  
    return x * x  
  
print(square(5))
```

2) Lambda Function

```
square = lambda x: x * x  
  
print(square(5))
```

Limitations of Lambda

- Only one expression
- Cannot contain loops
- Cannot contain multiple statements
- Cannot contain multiple return statements

3) map()

- map() applies a function to every element of an iterable and returns a map object.
- map() performs the same operation on every element of a collection.

- Without map

```
numbers = [1,2,3,4,5]

result = []

for i in numbers:
    result.append(i*i)

print(result)
```

- Using map

```
numbers = [1,2,3,4,5]

result = list(map(lambda x:x*x, numbers))

print(result)
```

4) filter()

- filter() selects only those elements that satisfy a given condition.
- filter() keeps only the elements for which the condition is True.

```
numbers = [10,15,20,25,30,35]

result = list(filter(lambda x:x%2==0, numbers))

print(result)
```

5) reduce()

- reduce() repeatedly applies a function to combine all elements of an iterable into a single value.
- reduce() reduces multiple values into one final result.

```
from functools import reduce

numbers = [1,2,3,4,5]

result = reduce(lambda x,y:x+y, numbers)

print(result)
```

- Difference Between map(), filter(), and reduce().

map()	filter()	reduce()
Transforms every element	Selects elements	Combines all elements
Returns all processed values	Returns only matching values	Returns one final value
Output is a collection	Output is a collection	Output is a single value